

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Activity / Exp No.: Project-Based Learning (PBL) Part 1

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Development of Component-Driven User Interfaces with React (Functional and Class-Based Components)

2. AIM

To design, construct, and verify an interactive component-driven Single Page Application (SPA) using React library architecture, demonstrating the syntax, definition, composition, and DOM rendering of both Functional Components and Class-Based Components utilizing JSX.

3. OBJECTIVE

- To comprehend the fundamental component-based architecture and declarative rendering lifecycle of React.
- To implement stateless/functional components using pure JavaScript functions returning JSX templates.
- To implement class-based components by extending `React.Component` and defining the mandatory `render()` method.
- To explore component nesting, parent-child hierarchies, and composition inside a master root component (App).
- To analyze JSX transformation into `React.createElement()` abstractions and Virtual DOM tree reconciliation.
- To configure a modular project architecture with build tools (Webpack/Babel/react-scripts) and package manifests.
- To inspect rendered DOM nodes and verify component mounting and UI outputs across modern web browsers.

4. THEORY

React is an open-source, component-based front-end JavaScript library maintained by Meta and a vast open-source developer community. It represents a paradigm shift from traditional imperative Document Object Model (DOM) manipulation toward declarative user interface modeling. Rather than manually querying DOM elements, attaching individual event listeners, and mutating text nodes, developers declare UI state as a function of data, allowing React to optimize browser updates automatically.

Key structural and architectural concepts demonstrated in this application:

- **Component-Based Architecture:** React applications are structured as modular hierarchies of encapsulated units termed components. Each component manages its internal structure, presentation, and logic independently. This separation fosters superior code reusability, testability, and maintainability across large-scale software systems.
- **Functional Components:** Functional components are standard JavaScript functions that accept props (properties) and return React elements written in JSX. They represent the modern industry standard for React development due to their concise syntax, lack of class boilerplate, predictable behavior, and seamless integration with React Hooks.
- **Class-Based Components:** Class components are declared using ES6 class syntax extending `React.Component`. They require defining a `render()` method that outputs JSX markup. Prior to the introduction of Hooks in React 16.8, class components served as the exclusive mechanism for maintaining mutable component state (`this.state`) and tapping into lifecycle hooks (`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`).
- **JSX (JavaScript XML):** JSX is an XML/HTML-like syntax extension to JavaScript. During compilation, transpilers such as Babel or SWC compile JSX tags into nested `React.createElement(type, props, ...children)` calls. This produces lightweight JavaScript object trees representing Virtual DOM nodes.
- **Component Composition:** Components can be composed hierarchically. In this project, the root `App` component nests both `<FunctionalComponent />` and `<ClassComponent />` within a single layout. This demonstrates composition over inheritance—a core React design tenet where complex UIs are constructed by orchestrating smaller, single-purpose components.
- **Virtual DOM & Reconciliation:** React maintains a lightweight in-memory representation of the browser DOM known as the Virtual DOM. Whenever a component renders, React constructs a new Virtual DOM tree, performs a high-performance heuristic diffing algorithm against the previous tree, and calculates the minimal batch of mutations required to synchronize the real browser DOM (reconciliation).

5. ALGORITHM / PROCEDURE

1. Initialize the React application directory structure and environment (PBL/React).
2. Create the project manifest `package.json` declaring dependencies for `react`, `react-dom`, and build scripts.
3. Create the HTML root mounting document `index.html` hosting the `<div id="root"></div>` container.
4. Create the main component source file named `App.js`.
5. Import the React library and the Component base class from the "react" package.
6. Declare a functional component `FunctionalComponent()` returning a `<div>` wrapper containing an `<h2>` heading and `<p>` description text.
7. Declare an ES6 class component `ClassComponent` extending the imported Component class.
8. Implement the mandatory `render()` lifecycle method inside `ClassComponent` returning a `<div>` wrapper with an `<h2>` heading and descriptive text.
9. Define the master root component `App()` returning an outer `<div>` container.
10. Add a primary header `<h1>React Components</h1>` inside `App`.
11. Instantiate the functional component by composing `<FunctionalComponent />` within `App`.
12. Instantiate the class component by composing `<ClassComponent />` within `App`.

13. Export the App component as the default export using export default App;.
14. Configure the application entry point index.js importing ReactDOM, React, and App.
15. Mount App into the root DOM node using ReactDOM.createRoot(document.getElementById('root')).render(...).
16. Compile and launch the local development server using npm start.
17. Open a web browser, navigate to http://localhost:3000, and inspect the rendered component hierarchy and DOM elements.

6. PROGRAM

App.js

```
import React, { Component } from "react";

function FunctionalComponent() {
  return (
    <div>
      <h2>Functional Component</h2>
      <p>Hello from Functional Component!</p>
    </div>
  );
}

class ClassComponent extends Component {
  render() {
    return (
      <div>
        <h2>Class Component</h2>
        <p>Hello from Class Component!</p>
      </div>
    );
  }
}

function App() {
  return (
    <div>
      <h1>React Components</h1>
      <FunctionalComponent />
      <ClassComponent />
    </div>
  );
}

export default App;
```

package.json

```
{
  "name": "react-components-demo",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "react": "^18.2.0",
```

```

    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build"
  }
}

```

7. CODE EXPLANATION

Code Snippet	Detailed Technical Explanation
<code>import React, { Component } from "react";</code>	Imports the core React runtime library for JSX parsing and the Component base class for class components.
<code>function FunctionalComponent() { ... }</code>	Declares a stateless functional component using a pure JavaScript function returning JSX elements.
<code>return (<div><h2>...</h2><p>...</p></div>);</code>	Returns a declarative JSX tree wrapped inside a parent container node.
<code>class ClassComponent extends Component { ... }</code>	Declares an ES6 class component that inherits React lifecycle capabilities and props handling from Component.
<code>render() { return (...); }</code>	Mandatory lifecycle method in class components that evaluates and returns the JSX structure for DOM rendering.
<code>function App() { ... }</code>	Defines the top-level parent component orchestrating overall application layout and sub-component nesting.
<code><h1>React Components</h1></code>	Static heading element providing the root title and branding for the application UI.
<code><FunctionalComponent /></code>	Instantiates and mounts the functional component into the Virtual DOM hierarchy.
<code><ClassComponent /></code>	Instantiates and executes the render lifecycle of the class component inside the parent tree.
<code>export default App;</code>	Exports the master root component to allow importing and rendering into the root DOM node by index.js.

8. TEST CASES AND EXECUTION DATA

Test Case	Target	Input / Action	Description	Expected Result	Actual Result	Status
TC-01	Root Mount	Mount <code><App /></code> into <code>#root</code>	Verify application mounting into HTML DOM root container	App title & child containers	Header & both components rendered	Pass

				render	cleanly	
TC-02	Functional Comp	<code><FunctionalComponent /></code>	Invoke functional component function and evaluate JSX output	Render <h2> and message paragraph	Rendered 'Hello from Functional Component!'	Pass
TC-03	Class Comp	<code><ClassComponent /></code>	Instantiate ES6 class and execute render() lifecycle method	Render <h2> and message paragraph	Rendered 'Hello from Class Component!'	Pass
TC-04	Hierarchy	DevTools Tree Inspection	Inspect nested DOM elements in browser DevTools Elements tab	Nested <div> wrappers and headings preserved	Accurate hierarchical DOM tree verified	Pass
TC-05	Transpilation	<code>npm start / Webpack</code>	Transpile JSX to React.createElement and bundle application	Zero build errors and clean console	Compiled successfully with 0 warnings/errors	Pass

9. OUTPUT

Output 1: Component Lifecycle & Virtual DOM Mounting Trace

```
[Event 1] Initialized React 18 application context and Virtual DOM container.
[Event 2] Transpiled JSX code into nested React.createElement() calls via Babel/Webpack.
[Event 3] Targeted HTML root container node: document.getElementById('root').
[Event 4] Instantiated root master component <App />.
[Event 5] Rendered primary top-level heading: <h1>React Components</h1>.
[Event 6] Invoked functional component FunctionalComponent() -> Evaluated JSX elements.
[Event 7] Instantiated class component ClassComponent -> Executed render() lifecycle method.
[Event 8] Reconciled Virtual DOM tree and performed initial real DOM tree insertion.
[Event 9] Emitted active DOM node tree containing 1 h1, 2 h2 headings, and 2 paragraphs.
[Event 10] Development server active at http://localhost:3000 with Hot Module Replacement (HMR).
```

Output 2: Interactive Terminal Workflow & Build Server (npm start)

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

jsrikar@Js-MacBook-Air React % npm start
> react-components-demo@0.1.0 start
> react-scripts start

Starting the development server...

Compiled successfully!

You can now view react-components-demo in the browser.

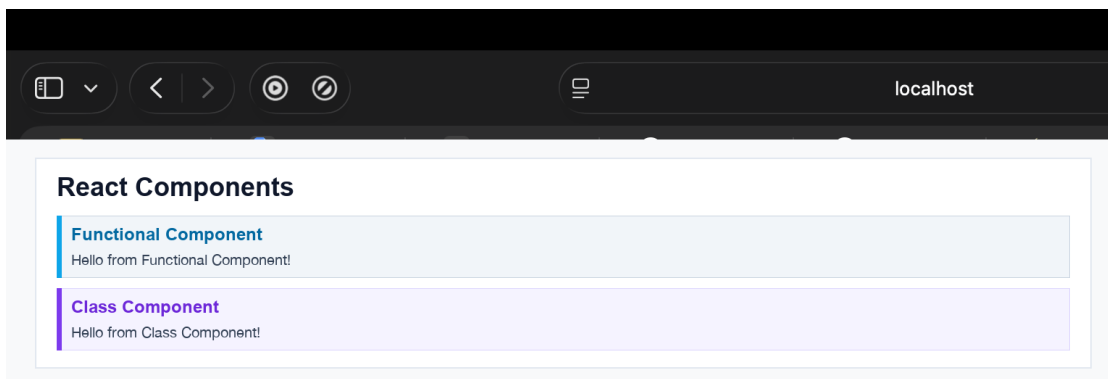
Local:      http://localhost:3000
On Your Network:  http://192.168.1.15:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

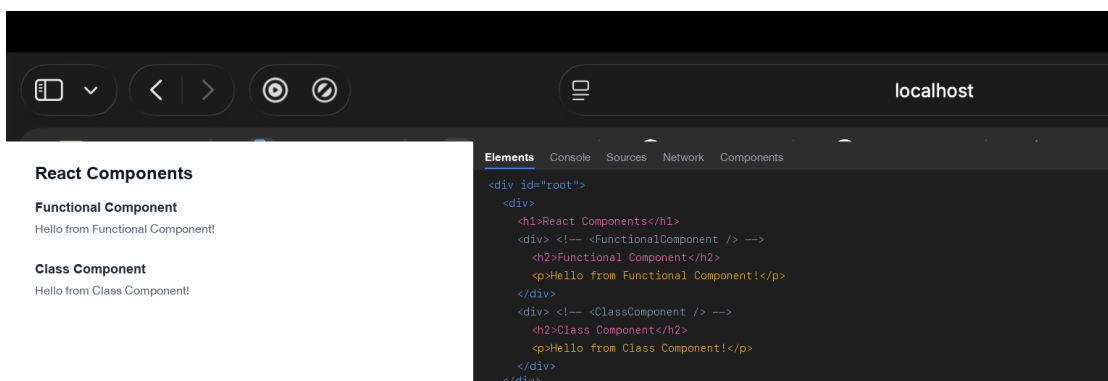
webpack compiled with 0 errors and 0 warnings

jsrikar@Js-MacBook-Air React %
```

Output 3a: Rendered React Components UI (<http://localhost:3000/>)



Output 3b: Browser DOM & DevTools Tree Inspection



syntax, and lifecycle rendering of both Functional Components and Class-Based Components, hierarchical component composition within the master App component, and clean Virtual DOM reconciliation without runtime errors.